

Testing mit Python

Oder: Wie man *etwas* besser programmiert

16. August 2023

1 Wieso sollte man seinen Code testen?

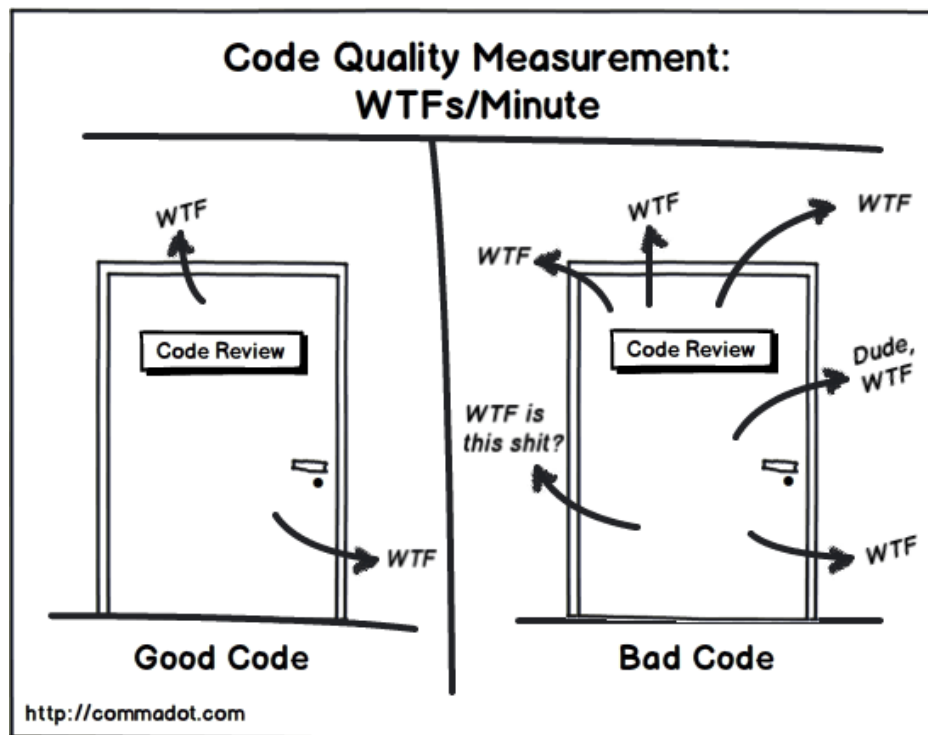
Die heutige Software-Landschaft ist geprägt von komplexen Systemen, die eine Vielzahl von Funktionen erfüllen und auf unterschiedlichen Plattformen und Geräten nahtlos zusammenarbeiten müssen. Mit dieser zunehmenden Komplexität wird die Sicherstellung der Zuverlässigkeit und Funktionalität von Software zu einer entscheidenden Herausforderung. Hier kommt das **Testen** ins Spiel - ein kritischer Bestandteil der Softwareentwicklung.

- **Qualitätssicherung des Codes:** Das Testen hilft sicherzustellen, dass der Code fehlerfrei funktioniert.
- **Vertrauen:** Gut getesteter Code schafft Vertrauen bei Entwicklern, Anwendern und Kunden.
- **Wartbarkeit:** Getesteter Code ist in der Regel leichter zu warten und zu erweitern.
- **Risikominderung:** Testen minimiert das Risiko von Fehlern in Produktionsumgebung.
- **Dokumentation:** Testfälle dienen als eine Art Dokumentation, die zeigt, wie der Code verwendet werden sollte.
- **Kostenersparnis:** Die frühzeitige Entdeckung und Behebung von Fehlern spart letztendlich Zeit und Geld im Entwicklungsprozess.

Es gilt immer:

Es gibt keinen fehlerfreien Code. Fehler sind unvermeidlich. Kein Code ist perfekt.

Testen hilft, Fehler zu finden, bevor sie zu ernsthaften Problemen werden.



1.1 Arten von Tests

- **Unit Tests:**

- Tests für kleinste, unabhängige Teile des Codes.
- Auf einzelne Funktionen, Methoden oder Klassen konzentrieren.
- Sicherstellen, dass diese Einheiten korrekt funktionieren.

- **Integrationstests:**

- Testen des Zusammenwirkens mehrerer Komponenten.
- Überprüfen, ob verschiedene Teile der Software gut zusammenarbeiten.
- Identifizieren von Problemen im Zusammenspiel der Komponenten.

- **Funktionstests:**

- Testen der Gesamtfunktionalität einer Anwendung.
- Benutzerszenarien simulieren, um das erwartete Verhalten sicherzustellen.

- **Akzeptanztests:**
 - Überprüfung, ob die Anwendung fachlichen Anforderungen erfüllt.
 - Validierung des Verhaltens aus Benutzersicht.
 - Sicherstellen, dass die Software den beabsichtigten Zweck erfüllt.
- **Lasttests:**
 - Simulation von Belastung und Beanspruchung der Software.
 - Überprüfung der Performance und Skalierbarkeit unter realen Bedingungen.
 - Identifizierung von Engpässen und Leistungsproblemen.
- **uvm...**

2 Testing Frameworks

In der Python-Welt gibt es verschiedene Frameworks für das Testen von Code, darunter `pytest` und `unittest`. Diese Frameworks werden verwendet, um automatisierte Tests zu erstellen und sicherzustellen, dass der Code wie erwartet funktioniert.

`unittest` ist das Testing-Framework, das in der Standard-Python-Bibliothek enthalten ist. Es bietet einen strukturierten Ansatz zum Schreiben von Tests und ist eng in die Python-Standardbibliothek integriert.

`pytest` ist ein leistungsfähiges und flexibles Test Framework von Drittanbietern. Es vereinfacht das Schreiben von Tests durch eine elegante und ausdrucksstarke Syntax. Pytest bietet auch viele zusätzliche Funktionen und Plugins für erweiterte Testmöglichkeiten.

Während `unittest` eher traditionell und in der Standardbibliothek verankert ist, zeichnet sich `pytest` durch seine Einfachheit, Flexibilität und Erweiterbarkeit aus. Die Wahl zwischen den beiden Frameworks hängt von den Präferenzen des Entwicklers und den Anforderungen des Projekts ab.

Kurz gesagt:

- **unittest:**

- Das in der Python-Standardbibliothek enthaltene Testframework.
- Unterstützt die Erstellung von Unit-Tests und Test-Suites.
- Ermöglicht die strukturierte Organisation von Tests.

- **pytest:**

- Beliebtes Drittanbieter-Testing-Framework.
- Bietet umfangreiche Funktionalitäten und Plugins.
- Automatische Entdeckung und Ausführung von Tests.
- Vereinfacht die Schreibweise von Tests durch eine erweiterte Syntax.

3 Die assert-Funktion im Testing

Die **assert**-Funktion ist ein fundamentales Konzept im Testen von Code in Python. Sie ermöglicht es, Bedingungen zu überprüfen und sicherzustellen, dass sie wahr sind. Wenn eine Bedingung nicht erfüllt ist, löst die **assert**-Funktion einen Fehler aus.

3.1 Verwendung von assert in unittest

In **unittest** wird die **assert**-Funktion in Testmethoden verwendet, um die erwarteten Ergebnisse von Funktionen zu überprüfen. Zum Beispiel:

```
import unittest

class MyTests(unittest.TestCase):

    def test_add(self):
        result = add(3, 5)
        self.assertEqual(result, 8)
```

3.2 Verwendung von assert in pytest

In **pytest** wird **assert** ähnlich verwendet:

```
def test_add():
    result = add(3, 5)
    assert result == 8
```

3.3 Wichtigste Befehle in Testing-Frameworks

Befehl	Beschreibung
<code>assertEqual(a, b)</code>	Überprüft, ob a gleich b ist
<code>assertNotEqual(a, b)</code>	Überprüft, ob a ungleich b ist
<code>assertTrue(x)</code>	Überprüft, ob x wahr ist
<code>assertFalse(x)</code>	Überprüft, ob x falsch ist
<code>assertIs(a, b)</code>	Überprüft, ob a dasselbe Objekt wie b ist
<code>assertIsNot(a, b)</code>	Überprüft, ob a nicht dasselbe Objekt wie b ist
<code>assertRaises(exception, func)</code>	Überprüft, ob func eine exception auslöst

4 unittest vs pytest

Um uns mit beiden Frameworks vertraut zu machen, haben wir zwei *fast* identische Funktionen geschrieben, von denen eine Fehler enthält:

```
# Funktion 1: Korrekte Addition
```

```
def add_correct(a, b):
    return a + b
```

```
# Funktion 2: Falsche Addition (mit einem Bug)
```

```
def add_with_bug(a, b):
    return [a - b] # Hier ist der Bug
```

Diese Funktionen werden wir benutzen mit dem Syntax von **unittest** und **pytest** zu spielen.

4.1 unittest

In Unittest (<https://docs.python.org/3/library/unittest.html>) muss für das Testen eine Klasse geschrieben werden. Es ist jedoch nicht notwendig, für jede Funktion eine eigene Klasse zu schreiben. Es ist jedoch üblich und empfehlenswert, Testklassen für verschiedene logische Einheiten oder Module

zu erstellen. Innerhalb dieser Testklassen können dann mehrere Testmethoden geschrieben werden, um verschiedene Aspekte dieser Einheit zu testen.

Syntax von unittest:

```
import unittest

class TestAddFunctions(unittest.TestCase):

    def test_add_correct_1(self):
        result = add_correct(3, 5)
        self.assertEqual(result, 8)

    def test_add_correct_2(self):
        result = add_correct(3, 5)
        self.assertIs(type(result), int)

    def test_add_with_bug_1(self):
        result = add_with_bug(3, 5)
        self.assertEqual(result, 8, f'Erwartet wird 8, haben {result} bekommen')

    def test_add_with_bug_2(self):
        result = add_with_bug(3, 5)
        self.assertIs(type(result), int, f'Erwartet wird int als Datentyp, haben {type(result)} bekommen')

class TestOtherFunctions(unittest.TestCase):

    def test_other_function_case_1(self):
        # ...

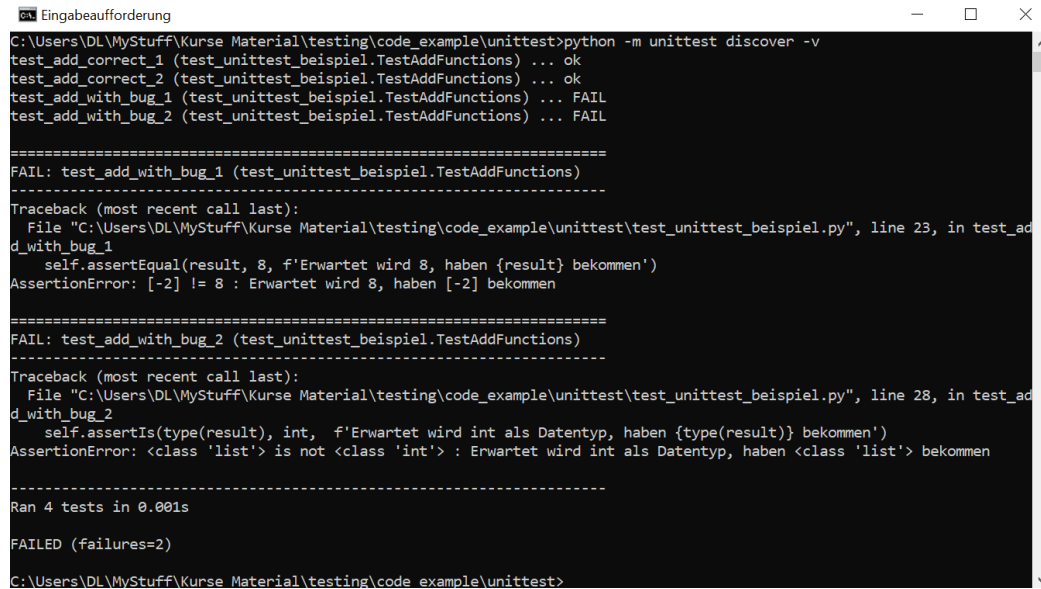
    def test_other_function_case_2(self):
        # ...

if __name__ == '__main__':
    unittest.main()
```

Jetzt können Tests für alle Dateien, die mit `test_` beginnen, in der Konsole gestartet werden:

```
python -m unittest discover -v
```

(der Parameter `-v` ist optional und sorgt dafür, dass mehr Informationen angezeigt werden):



```
C:\Users\DL\MyStuff\Kurse Material\testing\code_example\unittest>python -m unittest discover -v
test_add_correct_1 (test_unittest_beispiel.TestAddFunctions) ... ok
test_add_correct_2 (test_unittest_beispiel.TestAddFunctions) ... ok
test_add_with_bug_1 (test_unittest_beispiel.TestAddFunctions) ... FAIL
test_add_with_bug_2 (test_unittest_beispiel.TestAddFunctions) ... FAIL

=====
FAIL: test_add_with_bug_1 (test_unittest_beispiel.TestAddFunctions)
-----
Traceback (most recent call last):
  File "C:\Users\DL\MyStuff\Kurse Material\testing\code_example\unittest\test_unittest_beispiel.py", line 23, in test_add_with_bug_1
    self.assertEqual(result, 8, f'Erwartet wird 8, haben {result} bekommen')
AssertionError: [-2] != 8 : Erwartet wird 8, haben [-2] bekommen

=====
FAIL: test_add_with_bug_2 (test_unittest_beispiel.TestAddFunctions)
-----
Traceback (most recent call last):
  File "C:\Users\DL\MyStuff\Kurse Material\testing\code_example\unittest\test_unittest_beispiel.py", line 28, in test_add_with_bug_2
    self.assertIs(type(result), int, f'Erwartet wird int als Datentyp, haben {type(result)} bekommen')
AssertionError: <class 'list'> is not <class 'int'> : Erwartet wird int als Datentyp, haben <class 'list'> bekommen

-----
Ran 4 tests in 0.001s

FAILED (failures=2)

C:\Users\DL\MyStuff\Kurse Material\testing\code_example\unittest>
```

Wir können nun sehen, welche Tests fehlgeschlagen sind und die nächsten Schritte zur Reparatur des Codes einleiten. Zusätzlich erhalten wir auch `AssertionErrors`.

4.1.1 skip

In `unittest` gibt es verschiedene Methoden, um Tests zu überspringen oder zu deaktivieren, wenn bestimmte Bedingungen nicht erfüllt sind. Dies kann nützlich sein, wenn es Testfälle gibt, die nicht auf allen Systemen oder in allen Umgebungen ausgeführt werden sollen.

Der `@unittest.skipUnless` Dekorator erlaubt es, Tests zu überspringen, wenn eine bestimmte Bedingung nicht erfüllt ist. Zum Beispiel:

```
import unittest
import sys

class MyTests(unittest.TestCase):

    @unittest.skipUnless(sys.platform == "win32", "Nur
                        auf Windows")
    def test_windows_only(self):
        #Dieser Test wird nur auf Windows ausgeführt
```

Es gibt auch andere Möglichkeiten, Tests zu überspringen:

- `@unittest.skip`: Überspringt den Test unabhängig von einer Bedingung.
- `@unittest.skipIf`: Überspringt den Test, wenn eine bestimmte Bedingung erfüllt ist.
- `@unittest.expectedFailure`: Markiert einen Test als erwarteten Fehler, sodass er trotzdem läuft und im Falle eines Erfolgs als fehlgeschlagen markiert wird.

Das Überspringen von Tests ist nützlich, wenn du Tests auf bestimmten Plattformen, unter bestimmten Bedingungen oder in bestimmten Umgebungen ausführen möchtest. Es kann auch nützlich sein, wenn du an Tests arbeitest, die gerade fehlschlagen, aber du möchtest nicht, dass der Rest des Testlaufs davon beeinflusst wird.

Das Überspringen von Tests kann eine Alternative zum Mocken sein, wenn es sich um Tests handelt, die nicht deterministisch sind oder von externen Faktoren abhängen.

4.2 pytest

pytest (<https://docs.pytest.org/en/7.4.x/>) ist ein populäres und leichtgewichtiges Python-Testing-Framework, welches als Alternative zu `unittest` verwendet werden kann. Es bietet eine einfache und intuitive Syntax zum Schreiben und Ausführen von Tests. Im Vergleich zu `unittest` zeichnet sich `pytest` durch folgende Eigenschaften aus

- **Einfache Syntax:** Mit `pytest` kannst du Tests in einer sehr klaren und ausdrucksstarken Syntax schreiben. Dadurch können Tests schnell und effizient entwickelt werden.
- **Leichtgewichtig:** `pytest` benötigt weniger Boilerplate-Code im Vergleich zu `unittest`, was den Prozess des Testens erleichtert und beschleunigt.
- **Keine extra Klassen:** Im Gegensatz zu `unittest`, bei dem Testfälle in Klassen organisiert werden müssen, ermöglicht `pytest` das Schreiben von Tests ohne die Notwendigkeit von zusätzlichen Klassen.
- **Populärer in der Community:** Aufgrund seiner Einfachheit und Effizienz hat sich `pytest` zu einem der populärsten Testing-Frameworks in der Python-Community entwickelt.

Um `pytest` zu verwenden, installiere es mit dem Befehl:

```
pip install pytest
```

Danach kannst du Tests schreiben.

```
# test_my_functions.py
def add_correct(a, b):
    return a + b

def add_with_bug(a, b):
    return a - b # Falsche Operation, um einen Bug zu
                simulieren

def test_add_correct():
    result = add_correct(3, 5)
    assert result == 8

def test_add_with_bug():
    result = add_with_bug(3, 5)
    assert result == 8 # Hier wird der Fehler erwartet
```

Wenn wir Testing starten wollen, können wir das mit

`pytest -v`

tun. Wenn wir `pytest` ohne Dateinamen aufrufen, wird `pytest` automatisch nach Dateien suchen, die mit `test_` beginnen und die Endung `.py` haben. Diese Dateien sollten die Testfälle enthalten. Die Ausgabe sollte wie folgt aussehen:

```
C:\Users\DL\MyStuff\Kurse Material\testing\code_example>pytest -svv --capture=sys
===== test session starts =====
platform win32 -- Python 3.10.9, pytest-7.4.0, pluggy-1.0.0 -- C:\Users\DL\MyStuff\Anaconda\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\DL\MyStuff\Kurse Material\testing\code_example
plugins: anyio-3.5.0, typeguard-2.13.3
collected 2 items

test_pytest_beispiel.py::test_add_correct PASSED [ 50%]
test_pytest_beispiel.py::test_add_with_bug FAILED [100%]

===== FAILURES =====
test_add_with_bug

    def test_add_with_bug():
        result = add_with_bug(3, 5)
        > assert result == 8
E       assert [-2] == 8

test_pytest_beispiel.py:14: AssertionError
===== short test summary info =====
FAILED test_pytest_beispiel.py::test_add_with_bug - assert [-2] == 8
===== 1 failed, 1 passed in 0.16s =====

C:\Users\DL\MyStuff\Kurse Material\testing\code_example>
```

4.2.1 Parameter

In `pytest` können mit Hilfe von parametrisierten Tests mehrere `assert`-Überprüfungen in einem Testfall durchgeführt werden. Dies ermöglicht es, den gleichen Testfall mit unterschiedlichen Eingabewerten auszuführen und die erwarteten Ergebnisse zu überprüfen.

Der `@pytest.mark.parametrize` Dekorator wird verwendet, um einen Testfall zu parametrisieren. Hier ist ein Beispiel:

```
@pytest.mark.parametrize("test_input , expected", [(3, 5),
                                                    (6, 9), (15)])
def test_add_with_bug_with_parameter(test_input , expected):
    result = add_with_bug(test_input[0], test_input[1])
    assert result == expected
```

```
Eingabeaufforderung
C:\Users\DL\MyStuff\Kurse Material\testing\code_example\pytest>pytest -v
===== test session starts =====
platform win32 -- Python 3.10.9, pytest-7.4.0, pluggy-1.0.0 -- C:\Users\DL\MyStuff\Anaconda\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\DL\MyStuff\Kurse Material\testing\code_example\pytest
configfile: pytest.ini
plugins: anyio-3.5.0, typeguard-2.13.3
collected 5 items

test_pytest_beispiel.py::test_add_correct PASSED [ 20%]
test_pytest_beispiel.py::test_add_correct_datatype PASSED [ 40%]
test_pytest_beispiel.py::test_add_with_bug FAILED [ 60%]
test_pytest_beispiel.py::test_add_with_bug_with_parameter[test_input0-8] FAILED [ 80%]
test_pytest_beispiel.py::test_add_with_bug_with_parameter[test_input1-15] FAILED [100%]

===== FAILURES =====
test_add_with_bug

  def test_add_with_bug():
      result = add_with_bug(3, 5)
  >       assert result == 8
  E       assert [-2] == 8
test_pytest_beispiel.py:22: AssertionError

test_add_with_bug_with_parameter[test_input0-8]

test_input = (3, 5), expected = 8

  @pytest.mark.parametrize("test_input, expected", [(3, 5, 8), ((6, 9), 15)])
  def test_add_with_bug_with_parameter(test_input, expected):
      result = add_with_bug(test_input[0], test_input[1])
  >       assert result == expected
  E       assert [-2] == 8
test_pytest_beispiel.py:27: AssertionError

test_add_with_bug_with_parameter[test_input1-15]

test_input = (6, 9), expected = 15

  @pytest.mark.parametrize("test_input, expected", [(3, 5, 8), ((6, 9), 15)])
  def test_add_with_bug_with_parameter(test_input, expected):
      result = add_with_bug(test_input[0], test_input[1])
  >       assert result == expected
  E       assert [-3] == 15
test_pytest_beispiel.py:27: AssertionError

===== short test summary info =====
FAILED test_pytest_beispiel.py::test_add_with_bug - assert [-2] == 8
FAILED test_pytest_beispiel.py::test_add_with_bug_with_parameter[test_input0-8] - assert [-2] == 8
FAILED test_pytest_beispiel.py::test_add_with_bug_with_parameter[test_input1-15] - assert [-3] == 15
===== 3 failed, 2 passed in 0.10s =====
C:\Users\DL\MyStuff\Kurse Material\testing\code_example\pytest>
```

4.2.2 skip

In pytest gibt es zwei Möglichkeiten, Tests zu überspringen:

- Aufgrund von Bedingungen
- Aufgrund der Namen der Tests

Du kannst den `pytest.mark.skipif` Dekorator verwenden, um Tests basierend auf bestimmten Bedingungen zu überspringen. Es ist auch möglich, Tests aufgrund ihres Namens zu überspringen, indem wir den Parameter `-k` in der Kommandozeile verwenden: `pytest -v -k "not slow"`.

```

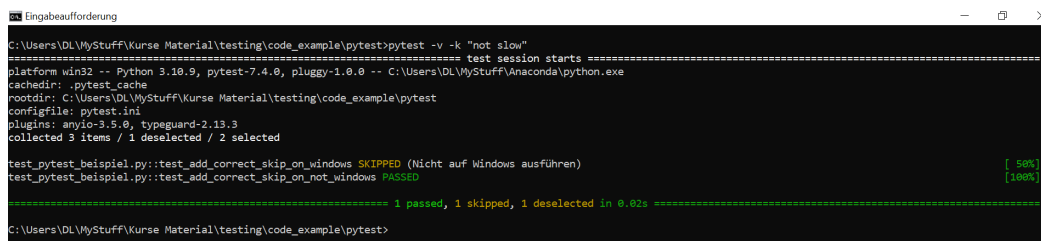
skip_on_windows = pytest.mark.skipif(sys.platform == "win32",
                                     reason="Nicht auf Windows ausfuehren")
skip_on_non_windows = pytest.mark.skipif(sys.platform != "win32",
                                     reason="Nur auf Windows ausfuehren")

@skip_on_windows
def test_add_correct_skip_on_windows():
    result = add_correct(3, 5)
    assert result == 8

@skip_on_non_windows
def test_add_correct_skip_on_not_windows():
    result = add_correct(3, 5)
    assert result == 8

@pytest.mark.slow
def test_add_correct_slow():
    result = add_correct(3, 5)
    time.sleep(5) # Simuliert eine langsame Operation
    assert result == 8

```



```

C:\Users\DL\MyStuff\Kurse Material\testing\code_example\pytest>pytest -v -k "not slow"
===== test session starts =====
platform win32 -- Python 3.10.9, pytest-7.4.0, pluggy-1.0.0 -- C:\Users\DL\MyStuff\Anaconda\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\DL\MyStuff\Kurse Material\testing\code_example\pytest
configfile: pytest.ini
plugins: anyio-3.5.0, typeguard-2.13.3
collected 3 items / 1 deselected / 2 selected

test_pytest_beispiel.py::test_add_correct_skip_on_windows SKIPPED (Nicht auf Windows ausführen) [ 50%]
test_pytest_beispiel.py::test_add_correct_skip_on_not_windows PASSED [100%]

===== 1 passed, 1 skipped, 1 deselected in 0.02s =====
C:\Users\DL\MyStuff\Kurse Material\testing\code_example\pytest>

```

Wir sehen, dass ein Test übersprungen wurde (Windows-Fall) und dass der lange Test komplett "deselektiert" wurde. Insgesamt bietet **pytest** flexible Möglichkeiten, Tests zu überspringen, sei es aufgrund von Bedingungen oder aufgrund des Testnamens.

5 "Best Practices" beim Testing

Beim Schreiben von Tests gibt es einige bewährte Praktiken, die helfen können, den Testprozess effizienter und effektiver zu gestalten.

5.1 Qualität über Quantität

Es ist wichtig, qualitativ hochwertige Tests zu schreiben, anstatt sich nur auf die Anzahl der Tests zu konzentrieren. Konzentriere dich auf die Erstellung aussagekräftiger Testfälle, die die Funktionalität und Korrektheit deines Codes umfassend überprüfen.

5.2 Gute Testabdeckung

Strebe eine angemessene Testabdeckung an, um sicherzustellen, dass verschiedene Pfade und Szenarien im Code getestet werden. Ziel ist es, potenzielle Bugs zu identifizieren und zu beheben, bevor sie zu Problemen in der Produktion führen.

5.3 Ordnerstruktur

Eine klare Ordnerstruktur für die Testdateien ist wichtig, um die Tests organisiert zu halten. Ein möglicher Ansatz ist:

```
project/  
|-- module/  
|   |-- module_code.py  
|-- tests/  
|   |-- test_module_code.py
```

In diesem Fall enthält der Ordner "tests" die Testdateien, die den entsprechenden Code in "module" testen.

5.4 Namenskonventionen

Verwende aussagekräftige Namen für deine Testfälle, damit ihre Bedeutung klar ist. Zum Beispiel beginnen Testnamen mit "test_" (z.B. 'test_functionality'), um ihre Funktion zu kennzeichnen.

5.5 Separation of Concerns

Trenne Tests für verschiedene Funktionen oder Module, um die Wartbarkeit zu verbessern. Jeder Test sollte sich auf eine bestimmte Funktionalität konzentrieren.

5.6 Mocks und Stubs

Verwenden Sie Mocks und Stubs, um externe Abhängigkeiten zu simulieren und isolierte Tests durchzuführen. Dadurch können die Auswirkungen externer Faktoren reduziert und die Zuverlässigkeit der Tests erhöht werden.

5.7 Kontinuierliches Testing

Integriere Tests in deinen Entwicklungsprozess und führe sie regelmäßig aus, um sicherzustellen, dass Änderungen keine unerwarteten Auswirkungen haben.

5.8 Testdatenmanagement

Verwende geeignete Testdaten, um verschiedene Szenarien zu überprüfen. Testdaten sollten vielfältig sein und auch Grenzfälle abdecken.

6 Abschlussbemerkung

Beim Testen von Python-Code stehen Entwicklern drei beliebte Bibliotheken zur Verfügung: `'unittest'`, `'pytest'` und `'nose'` (nicht weiter behandelt hier, siehe unten). Jede dieser Bibliotheken bietet leistungsstarke Funktionen, um sicherzustellen, dass der Code korrekt funktioniert und Fehler vermieden werden. Die Wahl zwischen diesen hängt oft vom persönlichen Geschmack und den Anforderungen des Projekts ab.

`'unittest'` bietet ein umfassendes Testframework, das in der Python Standardbibliothek enthalten ist. Es folgt dem xUnit-Stil und bietet eine Vielzahl von Funktionen, um Tests zu organisieren, auszuführen und zu überprüfen.

`'pytest'` ist für seine Einfachheit und Flexibilität bekannt. Es ermöglicht das Schreiben von weniger Code für umfangreiche Testabdeckung und bietet Funktionen wie Testparametrisierung, Testskipping und leistungsstarke Assertions. Es hat sich in der Entwicklergemeinschaft aufgrund seiner Benutzerfreundlichkeit und leistungsstarken Funktionen zunehmend durchgesetzt.

‘nose‘ ist eine weitere Alternative, die sich auf automatische Testererkennung und Testausführung spezialisiert hat. Obwohl es in der Vergangenheit populär war, ist seine Verwendung in der Community zurückgegangen, da ‘pytest‘ an Beliebtheit gewonnen hat.

Letztendlich liegt die Wahl zwischen ‘unittest‘, ‘pytest‘ und ‘nose‘ bei dir. Es lohnt sich, alle drei Bibliotheken auszuprobieren, um herauszufinden, welche am besten zu deinem Entwicklungsstil und den Anforderungen deines Projekts passt.